IBKR Import-Pipeline — Technische Dokumentation

Projekt: xml_proj — Trading-Journal Import-System **Stand:** 2026-05-20 (inkl. Running-Balance-Implementierung) **Ziel:** Vollständige Dokumentation des Datenflusses von IBKR-Rohdaten (XML/CSV) bis zur befüllten MySQL-Datenbank

---

Inhaltsverzeichnis

1. [Überblick & Ziele](#1-überblick--ziele)
2. [Datenquellen](#2-datenquellen)
3. [Pipeline-Architektur](#3-pipeline-architektur)
4. [Phase 0 — E-Mail-Abruf](#4-phase-0--e-mail-abruf-get_flexquery_emailspy)
5. [Phase 1 — Import (XML/CSV → trades_raw)](#5-phase-1--import-xmlcsv--trades_raw)
6. [Phase 2 — Processing (trades_raw → Endtabellen)](#6-phase-2--processing-trades_raw--endtabellen)
7. [Kernkonzepte](#7-kernkonzepte)
8. [Datenbankstruktur](#8-datenbankstruktur)
9. [Konfiguration & Multi-User](#9-konfiguration--multi-user)
10. [Fehlerbehandlung & Robustheit](#10-fehlerbehandlung--robustheit)
11. [Ausblick — Erweiterungen & Verbesserungen](#11-ausblick--erweiterungen--verbesserungen)

---

1. Überblick & Ziele

Was ist das System?

Das System liest automatisch **IBKR FlexQuery-Reports** (XML) und **Activity Statements** (CSV) ein und überführt die Rohdaten in ein strukturiertes Trading-Journal mit:

- **FIFO-Lot-Matching** für alle Assets (Aktien, Optionen, FX)
- **Kampagnen-Tracking** (gebündelte Trades je Symbol/Zeitraum)
- **Options-Strategie-Erkennung** (Spreads, Iron Condors, Straddles, ...)
- **Vollständigem Audit-Trail** (jede Aktion im `campaign_ledger`)
- **Multi-User/Multi-Environment-Support** (Echtgeld, Paper, Test, Automatisiert)

Warum zwei Phasen?

Die Trennung in **Import** (XML → `trades_raw`) und **Processing** (Matching + Kampagnen) ist bewusst gewählt:

Vorteil	Erklärung
Wiederholbarkeit	Import kann mehrfach ausgeführt werden (Checksums verhindern Doppelimport)
Debugging	`trades_raw` zeigt exakt was IBKR geliefert hat, unverändert
Trennung von Verantwortlichkeiten	Parser-Logik vs. Business-Logik klar getrennt
Datenbank-Reset	`reset_IBKR_DB.py` setzt **alle** Tabellen zurück (inkl. `trades_raw`) — Summary-Files ermöglichen den vollständigen Neuaufbau ohne Originaldateien

---

2. Datenquellen

2.1 FlexQuery XML

IBKR generiert **täglich** einen FlexQuery-Report als XML. Die Datei enthält alle Aktivitäten des Kontos für den gewählten Zeitraum.

Struktur:

```
<FlexQueryResponse>

  <FlexStatements>
    <FlexStatement accountId="U1234567" fromDate="..." toDate="...">
      <Trades>
        <Trade tradeID="..." symbol="AAPL" buySell="BUY" quantity="100" ... />
        <Trade tradeID="..." assetCategory="CASH" symbol="EUR.USD" ... />
      </Trades>
      <OptionEAE>
        <OptionEAE tradeID="..." transactionType="Expiration" ... />
      </OptionEAE>
      <CorporateActions>
        <CorporateAction type="RS" symbol="APDN" ... />  <!-- Reverse Split -->
      </CorporateActions>
    </FlexStatement>
  </FlexStatements>
</FlexQueryResponse>
```

Asset-Kategorien:

Code	Bedeutung	Beispiel
`STK`	Aktien	AAPL, TSLA
`OPT`	Optionen	AAPL 250117C00150000
`CASH`	FX-Trades	EUR.USD, GBP.USD

2.2 Activity Statement CSV

IBKR liefert täglich auch ein **Activity Statement CSV** — ein mehrteiliges Dokument mit Kontosnapshots (Cash, Net Liquidation, Margin, etc.). Es enthält **keine Einzeltrades**, sondern Kontostands-Daten.

Die CSV landet in einem **separaten Verzeichnis**: `C:\Trade_CSV\incoming\` (XML-Files hingegen kommen nach `C:\Trade_XML\incoming\`)

Beide Verzeichnisse werden automatisch über `get_flexquery_emails.py` befüllt (siehe Phase 0).

2.3 OptionEAE und BookTrades

IBKR erzeugt bei **automatischen Schließungsereignissen** (Verfall, Ausübung, Assignment) zwei Arten von Einträgen im XML:

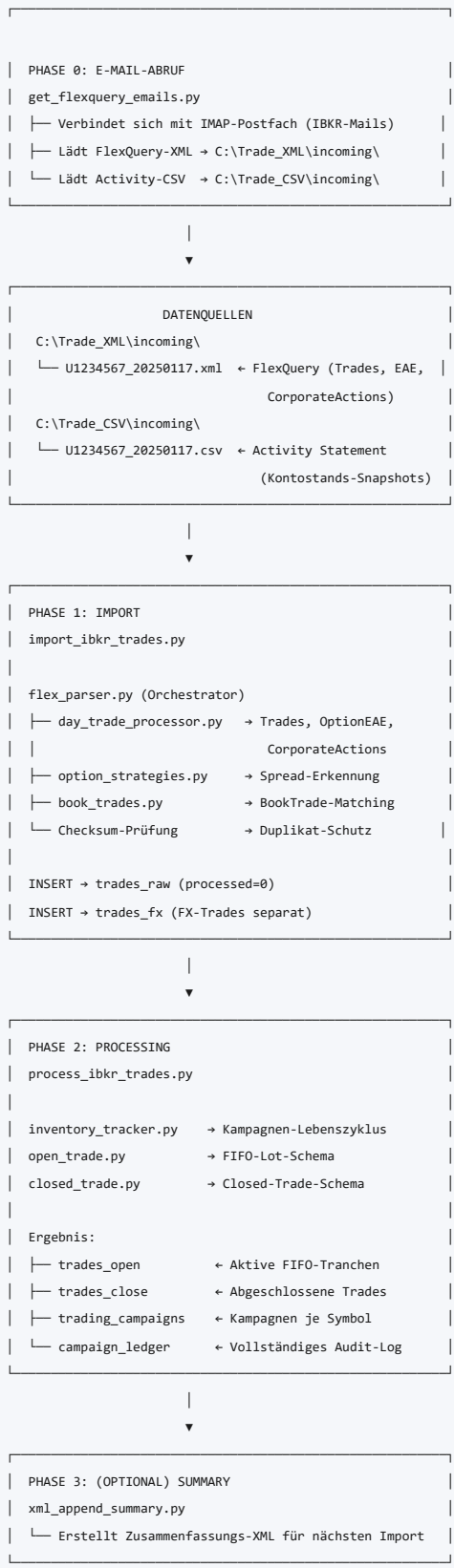
- **OptionEAE** (im `<OptionEAE>`-Block): Enthält das *Ereignis* mit Typ und Details (Expiration, Exercise, Assignment, CashSettlement). Aus diesem Block werden die Informationen gewonnen, um BookTrades korrekt zu klassifizieren.
- **BookTrade** (im `<Trades>`-Block, Flag `isBookTrade=1`): Der dazugehörige automatische *Trade-Eintrag*, den IBKR für die Buchung erzeugt. Allein stehend hat er noch keinen Status.

`book_trades.py` liest die OptionEAE-Daten und überträgt den jeweiligen Ereignistyp auf den passenden BookTrade — verbunden via `tradeID` oder `conid+date`.

---

3. Pipeline-Architektur

3.1 Gesamtübersicht



3.2 Pipeline-Runner

Alle Schritte werden über `run_pipeline.py` gesteuert:

```
python run_pipeline.py                    # alle Schritte

python run_pipeline.py --steps import     # nur Import
python run_pipeline.py --steps process    # nur Processing
python run_pipeline.py --env money        # nur Echtgeld
python run_pipeline.py --user mike        # nur User mike
```

`pipeline.toml` — Konfiguration:

```
[pipeline]

steps = ["fetch_emails", "import", "process", "summary"]


                ↑ Phase 0          ↑ Phase 1   ↑ Phase 2   ↑ Phase 3
```

```
[users.mike]
prefix = "mike_"
environments = ["money", "paper", "autom_money", "autom_paper"]



[users.karola]
prefix = "karola_"
environments = ["money"]
```

---

4. Phase 0 — E-Mail-Abruf (`get_flexquery_emails.py`)

IBKR stellt FlexQuery-Reports und Activity Statements **automatisiert per E-Mail** zu. `get_flexquery_emails.py` übernimmt den Abruf und die Verteilung in die richtigen Eingangsverzeichnisse.

```
get_flexquery_emails.py


|
├── Verbindet sich per IMAP mit dem konfigurierten Postfach (.env)
├── Durchsucht Inbox nach IBKR-Report-Mails (Betreff-Filter)
|
├── Für XML-Anhänge (FlexQuery-Report):
|   └── Speichert in C:\Trade_XML\incoming\
|       (enthält Trades, OptionEAE, CorporateActions)
|
├── Für CSV-Anhänge (Activity Statement):
|   └── Speichert in C:\Trade_CSV\incoming\
|       (enthält Kontostands-Snapshots, keine Einzeltrades)
|
└── Markiert verarbeitete Mails (gelesen / verschoben)
```

Konfiguration in `.env`:

```
EMAIL_HOST=imap.example.com


EMAIL_USER=trading@example.com
EMAIL_PASSWORD=...
EMAIL_FOLDER=INBOX
```

Wichtig: Phase 0 ist optional — XML/CSV-Dateien können auch manuell in die Eingangsverzeichnisse kopiert werden. Die nachfolgenden Phasen 1 und 2 sind davon unabhängig.

---

5. Phase 1 — Import (XML/CSV → trades_raw)

5.1 Einstiegspunkt: `import_ibkr_trades.py`

```
import_ibkr_trades.py


│
├── Für jede XML-Datei (aus C:\Trade_XML\incoming\):
│   ├── Lese <FlexStatement> (kann mehrere je Datei enthalten)
│   ├── Extrahiere accountId → route zu Environment/User (via config.py)
│   ├── Rufe flex_parser.process_flex_file() auf
│   └── Verschiebe Datei in processed/ oder deleted/
└── Scanne C:\Trade_CSV\incoming\ nach CSV-Dateien:
    └── activity_csv_parser → account_snapshots
```

Routing via `config.py`:

```
# Key = letzte 4 Ziffern der IBKR accountId

"U***9360" und "U12369360" ergeben denselben Key "9360"



ACCOUNT_MAP = {
    "9360": {"user": "mike",   "table_prefix": "mike_",   "environment": "money"},
    "8145": {"user": "karola", "table_prefix": "karola_", "environment": "money"},
}
```

Jede XML-Datei enthält immer nur Daten eines einzigen Kontos. Die `accountId` im `<FlexStatement>` bestimmt, welchem User und Environment die Trades zugeordnet werden.

5.2 Kern-Parser: `flex_parser.py`

Der Flex-Parser ist der **Orchestrator** der Import-Phase. Er koordiniert alle Sub-Parser.

```
process_flex_file(xml_path, db, repo, logger, environment)


│
├── 1. Extrahiere Rohdaten (parallel als DataFrames):
│   ├── extract_trades_for_db()          → df_trades
│   ├── extract_optioneae_for_db()       → df_eae
│   └── extract_corporateaction_for_db() → df_corp
│
├── 2. Aggregiere gleichartige Trades:
│   └── Gruppenbildung: (conid, symbol, buySell, ibOrderID)
│       Hintergrund: Exchange-Fills können in mehreren Teil-Trades kommen
│
├── 3. Options-Strategie-Erkennung:
│   └── option_strategies.detect_strategies(df_trades)
│
├── 4. BookTrade-Matching:
│   └── book_trades.handle_booktrades(df_trades, df_eae)
│
├── 5. CorporateActions sortieren:
│   └── sort_corporate_actions(df_corp)
│
├── 6. Checksum-Berechnung & Duplikat-Check:
│   ├── MD5(tradeID + datetime + symbol + qty + price)
│   └── SELECT checksums FROM trades_raw WHERE checksum IN (...)
│
├── 7. Symbol-Lookup/-Anlage:
│   └── get_symbols_bulk() → INSERT neue Instrumente
│
└── 8. INSERT in Datenbank:
    ├── INSERT INTO trades_raw (processed=0)
    └── INSERT INTO trades_fx
```

5.3 Element-Extraktion: `day_trade_processor.py`

Normale Trades (`extract_trades_for_db`):

Liest alle `<Trade>`-Elemente und mappt IBKR-Attribute auf interne Spaltennamen.

Sonderfall CASH-Trades (FX):

```
IBKR meldet einen EUR.USD-Trade als EINZELNEN Eintrag.


Das System erzeugt ZWEI Datenbankzeilen:
  ├── Zeile 1: EUR (Basiswährung) – positive Menge
  └── Zeile 2: USD (Quoting-Währung) – negative Menge (Gegenseite)
```

Dieses Design ermöglicht, FX-Trades wie normale Positionen per FIFO zu tracken.

OptionEAE-Extraktion (`extract_optioneae_for_db`):

`transactionType`	Bedeutung
`Expiration`	Option verfallen (wertlos)
`Exercise`	Option ausgeübt (durch Holder)
`Assignment`	Option zugewiesen (durch Writer)
`CashSettlement`	Barzahlung statt Delivery (z.B. SPX, XSP)

CorporateActions (`extract_corporateaction_for_db`):

IBKR-Type	Bedeutung
`RS`	Reverse Split
`SS`	Stock Split (Forward)
`FS`	Forward Split
`DI`	Dividende
`TC`	Takeover / Merger

5.4 Options-Strategie-Erkennung: `option_strategies.py`

IBKR liefert in den FlexQuery-Daten **keine Information über synthetische Produkte** wie Spreads oder andere Multi-Leg-Konstrukte — jedes Leg erscheint als eigenständiger, isolierter Trade ohne Verweis auf eine übergeordnete Strategie. Das System muss daher **heuristisch** erkennen, welche Einzeltrades zusammengehören.

Zwei Heuristiken werden kombiniert:

Methode 1: ibOrderID-Matching (primär)

```
IBKR weist zusammenhängenden Combo-Orders dieselbe ibOrderID zu.


→ Alle Legs mit gleicher ibOrderID = eine Strategie
```

Methode 2: 3-Sekunden-Zeitfenster (Fallback)

```
Für manuell gestaffelten Einstieg:


→ Alle Options-Trades desselben Kontos im 3s-Fenster
   mit passender Leg-Konfiguration = eine Strategie
```

Erkannte Strategien:

Name	Legs	Beschreibung
Bull Put Spread	2	Long Put (tiefer Strike) + Short Put (höherer Strike)
Bear Call Spread	2	Short Call (tiefer) + Long Call (höherer)
Iron Condor	4	Bull Put + Bear Call kombiniert
Straddle	2	Long/Short Call + Long/Short Put (gleicher Strike)
Strangle	2	Wie Straddle, aber unterschiedliche Strikes
Calendar Spread	2	Gleicher Strike, unterschiedliche Expiries
Butterfly	3	3 Strikes, mittlerer Strike doppelt gewichtet

Ergebnis: Jeder Leg erhält `opt_strategyID` (UUID) und `opt_strategyName` .

5.5 BookTrade-Klassifizierung: `book_trades.py`

```
Ausgangslage:
  • OptionEAE-Block enthält: Ereignistyp, conid, tradeID, Details
  • BookTrade im Trades-Block: automatischer Buchungseintrag, noch ohne Status



Klassifizierungs-Logik:
  Primär:   tradeID des BookTrade == tradeID im OptionEAE-Eintrag
            (für Assignment, Exercise, Expiration)
  Fallback: conid-Matching für Cash Settlement (kein direkter tradeID-Link),
            anschließend Datumsfilter zur Eingrenzung auf den richtigen Tag




Ergebnis:
  BookTrade erhält den Status aus dem OptionEAE-Ereignis:
  "Expired" / "Assigned" / "Exercised" / "CashSettled"
  (dieser Status wird in trades_close übernommen)
```

Sonderfall SPX / Cash-Settled Options:

```
SPX löst ZWEI OptionEAE-Einträge aus:


  1. Exercise (formaler Ereignistyp des Holders)
  2. CashSettlement (die eigentliche Barausgleich-Buchung)
→ Das System priorisiert CashSettlement als finalen Status am BookTrade
```

5.6 Duplikat-Schutz: Checksums

Konzept: Jeder Trade erhält vor dem Insert einen MD5-Hash:

```
checksum = MD5(tradeID + dateTime + symbol + quantity + price)
```

Ablauf:

```
1. Berechne Checksums für alle neuen Trades
1. SELECT checksums FROM trades_raw WHERE checksum IN (...)
2. Filtere bekannte Checksums heraus
3. INSERT nur wirklich neue Trades
```

Vorteil: Dasselbe XML kann mehrfach importiert werden (z.B. nach Neustart) — keine Duplikate.

---

6. Phase 2 — Processing (trades_raw → Endtabellen)

6.1 Einstiegspunkt: `process_ibkr_trades.py`

```
# Hole alle unverarbeiteten Trades (processed=0), chronologisch sortiert

trades = repo.get_unprocessed_trades(environment)




for trade in trades:
    process_raw_trade(logger, trade, repo, tracker, campaign_id)
    repo.mark_trade_processed("trades_raw", trade["id"])
```

Schlüsselprinzip: Trades werden in **Zeitreihenfolge** verarbeitet. FIFO ist nur korrekt, wenn ältere Positionen zuerst geschlossen werden.

6.2 FIFO-Matching: Die 6 Fälle

IBKR-Vorzeichenkonvention:

- **BUY:** `netCash` negativ (Geld fließt raus)
- **SELL:** `netCash` positiv (Geld fließt rein)

```
Aktueller Bestand im Tracker: current_qty

Neuer Trade:                  new_qty (positiv = Long, negativ = Short)




FIFO-Queue in trades_open (LIFO per Symbol, chronologisch)
```

Fall 0: CASH / FX-Trade

```
assetCategory = "CASH"


→ Keine Kampagne, kein FIFO
→ Aktion: FX_EXCHANGE im Ledger
→ INSERT in trades_fx (separat)
```

Fall 1: Gleiche Richtung (Positionsaufbau)

```
Bedingung: current_qty * new_qty > 0  ODER  current_qty = 0


Beispiel: Bestand = 100 AAPL, neuer Trade = BUY 50 AAPL
          → Bestand wird 150




Aktion: INSERT trades_open (status=OPEN)
        Kampagne: TRADE_OPEN im Ledger
```

Fall 2c: Exakter Gegenabschluss

```
Bedingung: abs(open_tranche.qty) == abs(new_qty), entgegengesetzt


Beispiel: Bestand = 100 AAPL (1 Tranche), neuer Trade = SELL 100 AAPL
          → Position wird 0




Aktion: INSERT trades_close (status=Closed)
        trades_open markieren: processed=1
        Kampagne: TRADE_CLOSE_FULL → Status PENDING
```

Fall 2a: Teilabschluss

```
Bedingung: abs(open_tranche.qty) > abs(new_qty), entgegengesetzt


Beispiel: Bestand = 100 AAPL (1 Tranche), neuer Trade = SELL 60 AAPL
          → 60 AAPL geschlossen, 40 AAPL bleiben offen




Aktion: INSERT trades_close (60 Stück, status=Closed)
        trades_open aktualisieren: 40 Stück bleiben (status=SPLIT_REST)
        Kampagne: TRADE_CLOSE_PARTIAL im Ledger
```

Schlüsselkonzept: Ratio-Skalierung

```
ratio = close_qty / open_qty  # z.B. 60/100 = 0.6
```

Alle Open-Trade-Werte werden mit ratio skaliert:

```
netCash_close   = open_trade.netCash   * ratio
commission_close = open_trade.commission * ratio
```

→ ROI und P&L sind korrekt trotz Teilabschluss

Fall 2b: Umkehrung (Reversal)

```
Bedingung: abs(new_qty) > abs(open_tranche.qty), entgegengesetzt

Beispiel: Bestand = 100 AAPL (mehrere Tranchen), neuer Trade = SELL 250 AAPL
          → Alle Tranchen schließen (FIFO), 150 AAPL werden Short

Aktion: Schleife über alle open_tranches bis remaining_qty = 0
        Remainder → INSERT als REV_OPEN
        Kampagne: neue Kampagne für Short-Position
```

Fall SPLIT: Corporate Action (Aktiensplit)

```
Bedingung: trade["is_split"] = True

Beispiel: APDN Reverse Split 1:25
          → Alle offenen Tranchen: qty / 25

Aktion: UPDATE trades_open SET quantity = quantity * factor
        UPDATE campaign inventory
        INSERT campaign_ledger (action=STOCK_SPLIT)
        Neue Tranchen-Preise: price * (1/factor)
```

6.3 Kampagnen-Lebenszyklus: `inventory_tracker.py`

Ursprung: Die Kampagne wurde konzipiert, um die **Wheel-Strategie** (auch "Bonus-Strategie") vollständig zu dokumentieren. Bei der Wheel-Strategie werden auf dasselbe Underlying zyklisch Cash-Secured Puts und Covered Calls verkauft — oft über Wochen oder Monate. Eine Kampagne bildet genau diesen Zyklus ab: vom ersten Put bis zur letzten Aktienlieferung oder Ausübung.

Die **10-Tage-Wartezeit** (PENDING-Phase) ist ebenfalls auf die Wheel-Strategie abgestimmt: Nach dem scheinbaren Ende eines Zyklus (Position = 0) kann innerhalb weniger Tage ein neuer Put auf dasselbe Underlying eröffnet werden. Die PENDING-Phase verhindert, dass dafür sofort eine neue Kampagne angelegt wird — stattdessen wird der Trade in die bestehende aufgenommen, solange das Zeitfenster noch offen ist. Der Wert ist parametrierbar (`MAX_GAP_DAYS` in der Konfiguration).

Konzept: Eine *Kampagne* fasst alle Trades zu einem Symbol in einem zusammenhängenden Zeitraum zusammen.

```
AKTIVE Kampagne:

  ├── Symbol: AAPL
  ├── Start: 2025-01-10
  ├── Bestand: 150 Aktien + 2 Calls
  ├── Status: ACTIVE
  └── Ledger: TRADE_OPEN (x3), TRADE_CLOSE_PARTIAL (x1)



Nach vollständigem Schließen:
  └── Status: PENDING (Wartezeit: 10 Tage)
      → Sicherheitsnetz: Falls IBKR nachträgliche Korrekturtrades sendet



Nach 10 Tagen ohne Aktivität:
  └── Status: CLOSED
      ├── closed_at: 2025-02-05
      └── Finale P&L berechnet
```

Ledger-Aktionen:

Action	Auslöser
`TRADE_OPEN`	Neue Position (Fall 1)
`TRADE_CLOSE_FULL`	Vollständiges Schließen (Fall 2c)
`TRADE_CLOSE_PARTIAL`	Teilabschluss (Fall 2a)
`OPTION_EXPIRATION`	OptionEAE: Verfall
`OPTION_EXERCISE`	OptionEAE: Ausübung
`OPTION_ASSIGNMENT`	OptionEAE: Assignment
`CASH_SETTLEMENT`	OptionEAE: Barausgleich
`FX_EXCHANGE`	CASH-Trade
`STOCK_SPLIT`	CorporateAction: Split

6.4 Schema-Validierung: `open_trade.py` / `closed_trade.py`

Warum Schemas?

Ohne explizite Validierung können falsche Datentypen, fehlende Felder oder falsche Vorzeichen unbemerkt in die Datenbank gelangen und P&L-Berechnungen verfälschen.

`OpenTradeSchema.validate()` :

```
REQUIRED_FIELDS = ["quantity", "tradePrice", "netCash", "status", ...]

FIELD_TYPES = {"quantity": Decimal, "tradePrice": Decimal, ...}



## Status-Validierung:



assert status in [TradeStatus.OPEN, TradeStatus.REV_OPEN, TradeStatus.SPLIT_REST]
```

`to_closed_trade(open_trade, close_data, ratio)` :

Dies ist die **kritischste Funktion** im System. Sie konvertiert eine offene Tranche in einen abgeschlossenen Trade:

```
# Open-Seite (skaliert mit ratio):

netCash_open = open_trade.netCash * ratio
commission_open = open_trade.commission * ratio
```

Close-Seite (vom neuen Trade):

```
netCash_close = close_data.netCash * ratio
```

P&L-Berechnung:

```
profit = netCash_open + netCash_close - commission_open - commission_close
```

(netCash_open ist negativ bei Long-Kauf → profit steigt wenn netCash_

ROI-Berechnung:

```
roi = profit / abs(netCash_open)  # Basis: investiertes Kapital
```

Haltezeit:

```
days_held, duration_text = calculate_hold_metrics(open_dt, close_dt)
```

---

7. Kernkonzepte

7.1 Warum FIFO?

IBKR-Steuerberichte verwenden FIFO (First In, First Out) für Lot-Matching — das System bildet diese Logik nach, damit P&L-Berechnungen mit offiziellen IBKR-Berichten übereinstimmen. Die FIFO-Queue in `trades_open` ist nach `datetime` aufsteigend sortiert: älteste Tranche wird zuerst geschlossen.

Einschränkung beim Automations-Trading (TAT): Der automatisierte Trader (TAT) verwaltet intern seine eigene Lot-Zuordnung und schließt Positionen nicht zwingend in FIFO-Reihenfolge — insbesondere wenn mehrere offene Tranchen denselben Strike haben. In diesen Fällen kann die FIFO-Zuordnung dieses Systems von der TAT-internen Zuordnung abweichen: P&L und Haltezeit einzelner Trades werden dann anders berechnet als TAT es intern ausweist, auch wenn das Gesamtergebnis der Kampagne übereinstimmt.

7.2 Warum `Decimal` statt `float`?

```
# float-Problem:

0.1 + 0.2 == 0.3  # → False (0.30000000000000004)
```

In der Finanzwelt ist das inakzeptabel:

Summe aus 1000 Trades kann um Cent-Beträge abweichen

Lösung: Python Decimal (exakte Dezimalarithmetik)

```
from decimal import Decimal
to_decimal("0.1") + to_decimal("0.2") == Decimal("0.3")  # → True
```

`to_decimal()` aus `trading_common` ist robust gegen verschiedene Eingabeformate:

- Strings mit Tausenderpunkten (`"1.234,56"`)
- Float (konvertiert über String, vermeidet Float-Fehler)
- None / leer → `Decimal(0)`

7.3 Multi-User / Multi-Environment

Das System verwaltet bis zu 5 Environments je User:

Environment	Datenbank (Default)	Zweck
`money`	`ibkr_money`	Echtes Kapital
`paper`	`ibkr_paper`	Papierkonto / Simulation
`test`	`ibkr_test`	Entwicklung und Tests
`autom_money`	`ibkr_autom_money`	Automatisiertes Echtgeld
`autom_paper`	`ibkr_autom_paper`	Automatisiertes Paper

DB-Namen können per `.env` überschrieben werden (`DB_NAME_MONEY` , `DB_NAME_PAPER` , ...).

Tabellen-Präfix (`TABLE_PREFIX` aus `.env`):

```
-- User "mike" mit prefix "mike_":

SELECT * FROM mike_trades_raw;
SELECT * FROM mike_trading_campaigns;




-- User "karola" mit prefix "karola_":
SELECT * FROM karola_trades_raw;
```

Dies erlaubt mehrere User in **einer MySQL-Datenbank** ohne Kollisionen.

7.4 Transaktionen und Atomizität

Jeder Trade wird innerhalb einer Datenbank-Transaktion verarbeitet:

```
with db.transaction():


    # Alle INSERTs/UPDATEs für einen Trade
    open_id = repo.insert_open_trade(trade_dict)
    repo.link_to_campaign_new(campaign_id, ...)
    repo.mark_trade_processed("trades_raw", raw_id)
    # Bei Fehler: Rollback → keine halbfertigen Zustände
```

Ohne Transaktionen könnten bei einem Absturz inkonsistente Zustände entstehen (z.B. `trades_close` vorhanden, aber `trades_open` nicht als processed markiert).

---

8. Datenbankstruktur

8.1 Tabellen-Übersicht

```
trades_raw        ← Rohdaten (temporärer Puffer)

    │
    ▼
trades_open       ← Aktive FIFO-Tranchen
    │
    ├──→ trades_close      ← Abgeschlossene Trades (P&L hier)
    │
    └──→ trading_campaigns ← Kampagne je Symbol
              │
              └──→ campaign_ledger  ← Vollständiger Audit-Trail




symbols           ← Instrument-Referenzdata (conid, ISIN, ...)
trades_fx         ← FX-Trades (separat)
account_snapshots ← Kontostand-Snapshots (aus CSV)
```

8.2 Schlüsseltabellen im Detail

`trades_raw` — Rohdaten-Puffer

```
id, checksum,                       -- Eindeutigkeit


accountId, environment,             -- Routing
symbol, assetCategory, buySell,     -- Instrument
quantity, tradePrice, netCash,      -- Mengen/Preise
ibOrderID, tradeID,                 -- IBKR-Referenzen
opt_strategyID, opt_strategyName,   -- Strategie-Info
is_split, is_booktrade,             -- Sondertypen
processed,                          -- Status (0/1)
dateTime, settleDate                -- Zeitstempel
```

`trades_open` — Aktive Tranchen

```
id, campaign_id,


symbol, assetCategory,
quantity, tradePrice, netCash,      -- Einstiegswerte
ibCommission,
status,                             -- OPEN | REV_OPEN | SPLIT_REST
opt_strategyID, opt_strategyName,
dateTime, settleDate,
source_raw_id                       -- Rückverweis auf trades_raw
```

`trades_close` — Abgeschlossene Trades

```
id, campaign_id,

-- Open-Seite:
tradeIDOpen, tradePriceOpen, dateTimeOpen, netCashOpen, quantityOpen,
-- Close-Seite:
tradeIDClose, tradePriceClose, dateTimeClose, netCashClose, quantityClose,
-- Ergebnis:
profit, profitEUR, roi,
daysHeld, durationText,
commission, commissionEUR,
status,                              -- Closed | Expired | Assigned | CashSettled
opt_strategyID, opt_strategyName
```

`trading_campaigns` — Kampagnen

```
id, user, environment,

symbol, assetCategory,
status,                              -- ACTIVE | PENDING | CLOSED
start_date, end_date, closed_at,
total_profit, total_commission,
inventory                            -- Aktueller Bestand
```

`campaign_ledger` — Audit-Trail (ES-Lite)

```
id, campaign_id,

action,                              -- TRADE_OPEN | TRADE_CLOSE_FULL | ...
inventory_delta,                     -- Bestandsänderung
netcash, proceeds, commission,       -- Cashflows dieser Zeile
realized_pnl,                        -- P&L dieses Events
cum_pnl,                             -- kumulierter P&L der Kampagne bis hier
cum_commission,                      -- kumulierte Kommissionen bis hier
cum_premium,                         -- kumulierte Erlöse bis hier
source_table, source_id,             -- Rückverweis (TRADES_OPEN / TRADES_CLOSE / TRADES_RAW)
dateTime
```

Die drei `cum_*`-Spalten ermöglichen Zeitreise-Abfragen ohne vollständigen Replay:

```
-- P&L-Stand der Kampagne am 15. März:

SELECT cum_pnl FROM mike_campaign_ledger
WHERE campaign_id = 42 AND dateTime <= '2025-03-15'
ORDER BY dateTime DESC, id DESC LIMIT 1;
```

8.3 Running Balance — ES-Lite-Upgrade

Das `campaign_ledger` wurde um drei kumulierte Salden erweitert, die bei jedem INSERT automatisch berechnet und mitgeschrieben werden (`inventory_tracker.py → _get_running_totals()`):

Spalte	Berechnung	Nutzen
`cum_pnl`	SUM(realized_pnl) je Kampagne	historischer P&L-Stand
`cum_commission`	SUM(commission) je Kampagne	Gesamtkosten-Verlauf
`cum_premium`	SUM(proceeds) je Kampagne	Prämieneinnahmen (Wheel-Tracking)

Bestehende Daten wurden per `src/migrate_running_balance.py` nachbefüllt (Window-Functions):

```
python -m src.migrate_running_balance                # alle Environments

python -m src.migrate_running_balance money mike_    # einzelnes Target
```

---

9. Konfiguration & Multi-User

9.1 Konfigurationsdateien

Datei	Zweck	Versioniert?
`.env`	Secrets (DB-Passwort, API-Keys)	**Nein** (gitignore)
`config.py`	Account-Mapping, DB-Konfiguration	Ja
`pipeline.toml`	Pipeline-Schritte, aktive User	Ja
`CLAUDE.md`	Entwickler-Dokumentation	Ja

9.2 `.env`-Struktur (Auszug)

```
DB_HOST=localhost

DB_PORT=3306
DB_USER=root
DB_PASSWORD=...
TABLE_PREFIX=mike_




IBKR_FLEX_TOKEN=...
EMAIL_HOST=imap.example.com
EMAIL_USER=...
EMAIL_PASSWORD=...
```

9.3 DB-Engine-Pattern (`src/config.py`)

```
from src.config import get_engine

# Environment bestimmt Datenbank-Name (aus DB_MAP + .env):

engine = get_engine("money")

# → mysql+pymysql://user:pass@localhost:3306/ibkr_money
```

MySQL-Modus für Windows 11 Kompatibilität — wird bei jeder Verbindung gesetzt:

```
# Verhindert MySQL-Fehler bei NULL-Datumswerten (Win11 ist strenger als Win10)

SET SESSION sql_mode='NO_ZERO_IN_DATE,NO_ZERO_DATE,NO_ENGINE_SUBSTITUTION'
```

---

10. Fehlerbehandlung & Robustheit

10.1 Bekannte Bugfixes (Lernfälle)

Split-Bug: Kampagne bleibt ACTIVE nach Split

```
Problem: Nach RS/SS/FS war split_symbol = trade.get('underlyingSymbol')

         → underlyingSymbol enthielt "APDN.OLD" (altes Symbol)
         → Inventory wurde nie auf 0 gesetzt
         → Kampagne blieb ACTIVE




Fix: split_symbol = trade.get('symbol')
     (nach _handle_split_trade enthält symbol bereits das neue Symbol)
```

Enum-Bug: str(TradeStatus.OPEN) → "TradeStatus.OPEN"

```
Problem: Python 3.11+ änderte str()-Verhalten für str-Enums

         str(TradeStatus.OPEN) → "TradeStatus.OPEN" statt "OPEN"
         → Datenbankvergleiche schlugen fehl




Fix: raw.value if hasattr(raw, 'value') else str(raw)
     Merke: Nach Enum-Refactorings immer alle str(enum_val)-Vergleiche prüfen
```

P&L-Skalierungsfehler bei Teilabschluss

```
Problem: commission wurde nicht mit ratio skaliert

         → Bei Teilabschluss wurde volle Kommission dem Close zugerechnet




Fix: commission_close = open_trade.commission * ratio
```

10.2 Duplikat-Schutz

- **Checksum-basiert:** Verhindert Doppel-Import derselben XML
- `processed`**-Flag:** Verhindert Doppel-Processing desselben Trades
- `campaign_ledger`**-Atomizität:** Kein Ledger-Eintrag ohne dazugehörigen Trade

10.3 Logging

Jede Phase schreibt in `log/` mit Timestamp. Logger-Ebenen:

```
DEBUG   → detaillierte FIFO-Schritte


INFO    → Trade-Ergebnisse, Kampagnen-Status
WARNING → fehlende Daten, unerwartete Fälle
ERROR   → Fehler mit Stacktrace (kein Absturz der Pipeline)
```

---

11. Ausblick — Erweiterungen & Verbesserungen

11.1 Kurzfristige Verbesserungen

A) Streaming-Processing statt Batch

```
Problem:   Alle Trades werden in RAM geladen (DataFrame)


Risiko:    Bei sehr großen Historien (10.000+ Trades) hoher Speicherbedarf
Lösung:    Generator-basiertes Streaming – trades_raw zeilenweise verarbeiten
           (yield-Pattern bereits in Teilen vorhanden)
```

B) Automatische Retry-Logik

```
Problem:   Bei Datenbankfehler mid-Processing: unklar welche Trades schon verarbeitet


Lösung:    Idempotente Operationen + Retry-Counter in trades_raw
           (max_retries=3 → FAILED-Status nach Erschöpfung)
```

C) Validierungs-Report beim Import

```
Aktuell:   Fehler landen im Log, keine strukturierte Auswertung

Verbesserung:


- Anzahl importierter / übersprungener / fehlerhafter Trades als JSON
- Webhook-Notification bei Fehlern (Slack, E-Mail)
```

11.2 Mittelfristige Erweiterungen

D) Broker-Agnostik: Weitere Datenquellen

```
Problem:   System ist auf IBKR FlexQuery fixiert

Potential: Andere Broker (Tastyworks, TD Ameritrade) über Adapter anbinden




Ansatz: BrokerAdapter-Interface
  ├── IBKRFlexAdapter (bestehend)
  ├── TastyworksAdapter (CSV-Format)
  └── GenericCSVAdapter (konfigurierbar)




Der Parser-Orchestrator (flex_parser.py) ruft nur das Interface auf.
```

E) Real-Time-Import via IBKR TWS API

```
Aktuell:   Täglicher Batch-Import via E-Mail

Potential: Live-Import via IBKR TWS API (Interactive Brokers API)
           → Trades werden sekündlich importiert
           → Dashboard zeigt Live-P&L




Anforderungen:


- ibapi Python-Paket
- Separater Service-Prozess (always-on)
- trades_raw als Puffer bleibt bestehen
```

F) Erweitertes Strategie-Tracking

```
Aktuell:   Strategie-Name wird beim Import gesetzt und bleibt statisch

Potential:


- Strategie-Zustand über Laufzeit tracken (z.B. IC: welcher Flügel getroffen?)
- Automatisches Rollover-Erkennen (gleiche Strategie, neue Expiry)
- Greeks-Snapshot bei Eröffnung (via TWS API)
```

G) Steuer-Export

Potential: Automatischer Export als:

- FIFO-Steuerreport (Österreich/Deutschland)
- CSV für Steuerberater
- Vergleich mit IBKR Annual Tax Report

Basis: trades_close enthält alle nötigen Daten

11.3 Langfristige Architektur-Ideen

H) Event-Sourcing statt FIFO-Matching on-the-fly

```
Aktuelles Modell:

  trades_raw → FIFO-Matching → trades_open + trades_close



Event-Sourcing-Ansatz:
  Alle Ereignisse (TradeOpened, TradeClosed, PositionSplit, ...) in
  einer "events"-Tabelle speichern.
  Aktueller Zustand = Replay aller Events.



Vorteil: Zeitreise möglich (wie sah P&L am 15.03. aus?)
         Kampagnen rückwirkend neu berechnen ohne Datenverlust
Nachteil: Komplexer, langsamer Read-Pfad
```

I) Plugin-System für Corporate Actions

```
Aktuell:   Splits werden hardcoded behandelt

Potential: Jeder CorporateAction-Typ = eigener Handler (Plugin)
           → einfaches Erweitern ohne Kerncode zu ändern



class CorporateActionHandler(ABC):
    @abstractmethod
    def handle(self, action, repo, tracker): ...



class SplitHandler(CorporateActionHandler): ...
class DividendHandler(CorporateActionHandler): ...
class MergerHandler(CorporateActionHandler): ...
```

J) Tagesabschluss-Snapshot

Potential: Tägliche Bestandsaufnahme als separater DB-Eintrag:

- Open P&L je Symbol (unrealisiert)
- Margin-Verbrauch
- Greeks-Summe (Delta, Theta, Vega)

Basis: Kombination aus trades_open + account_snapshots + TWS API

Anhang: Technologie-Stack

Komponente	Technologie	Version
Sprache	Python	3.14
Datenbank	MySQL	8.4
ORM/DB-Driver	SQLAlchemy + PyMySQL	2.0.x + 1.1.x
Daten-Manipulation	pandas	aktuelle
Dezimal-Arithmetik	Python `decimal`	stdlib
XML-Parsing	Python `xml.etree.ElementTree`	stdlib
Konfiguration	python-dotenv	aktuelle
Pipeline-Config	TOML	stdlib (Python 3.11+)
Shared Utilities	`trading_common` (pip install -e)	intern
Tests	pytest	aktuelle

Anhang: Wichtige Dateipfade

```
xml_proj/

├── run_pipeline.py              ← Einstiegspunkt (orchestriert alle Phasen)
├── pipeline.toml                ← Pipeline-Konfiguration (User, Environments, Steps)
├── src/
│   ├── get_flexquery_emails.py  ← Phase 0: IMAP-Abruf → Trade_XML / Trade_CSV
│   ├── import_ibkr_trades.py    ← Phase 1: XML/CSV-Import → trades_raw
│   ├── process_ibkr_trades.py   ← Phase 2: FIFO-Processing → Endtabellen
│   ├── xml_append_summary.py    ← Phase 3: Summary-XML für DB-Rebuild
│   ├── migrate_running_balance.py ← Migration: cum_pnl/cum_commission/cum_premium
│   ├── import_activity_statement.py ← CSV-Snapshot-Import (account_snapshots)
│   ├── config.py                ← get_engine(), ACCOUNT_MAP, DB_MAP, TABLE_PREFIX
│   ├── trade_enums.py           ← TradeStatus, AssetCategory, CampaignStatus
│   ├── parsers/
│   │   ├── flex_parser.py       ← XML-Orchestrator (process_flex_file)
│   │   ├── day_trade_processor.py  ← Element-Extraktion (Trades, EAE, CorpActions)
│   │   ├── option_strategies.py ← Strategie-Erkennung (ibOrderID + 3s-Fallback)
│   │   ├── inventory_tracker.py ← Kampagnen-Lifecycle + Running Balance
│   │   ├── book_trades.py       ← BookTrade-Klassifizierung via OptionEAE
│   │   └── activity_csv_parser.py ← Activity-Statement-CSV-Parser
│   ├── trade_schemas/
│   │   ├── base_schema.py       ← Abstrakte Basis (validate, REQUIRED_FIELDS)
│   │   ├── open_trade.py        ← OpenTradeSchema + to_closed_trade() + ratio-Skalierung
│   │   └── closed_trade.py      ← ClosedTradeSchema
│   ├── db/
│   │   └── db_access.py         ← DBAccess (SQL) + TradeRepository (Domain-Queries)
│   └── utils/
│       ├── div_utils.py         ← to_decimal(), compute_checksum(), calculate_hold_metrics()
│       ├── corporate_utils.py   ← SPLIT_TYPES (RS, SS, FS)
│       └── path_utils.py        ← get_project_root()
├── tools/
│   └── maintenance/
│       └── reset_IBKR_DB.py     ← DB-Reset (destruktiv! alle Tabellen inkl. trades_raw)
├── sql/                         ← DB-Schema-Dateien (__PREFIX__ Platzhalter)
│   ├── trades_raw.sql
│   ├── trades_open.sql
│   ├── trades_close.sql
│   ├── trading_campaigns.sql
│   └── campaign_ledger.sql      ← inkl. cum_pnl, cum_commission, cum_premium
├── docs/
│   ├── pipeline-workflow.md     ← diese Dokumentation
│   └── pipeline-workflow.pdf    ← PDF-Export
├── tests/                       ← pytest (DummyLogger, DummyDBAccess)
├── input/                       ← Test-XML/CSV-Dateien
└── log/                         ← Anwendungslogs (RotatingFileHandler)
```